

Handout - Hausaufgabe 2

Fedir Deineko, Alexander Huhn

12 Dezember 2025

Inhaltsverzeichnis

1	Einführung und Motivation	2
2	Theoretische Grundlagen	2
2.1	Zweite finite Differenz	2
2.2	Diskretisierung des Gebietes	3
2.3	Diskretisierung des Laplace-Operators	3
2.4	Definition der Block-Diagonal-Matrizen	4
2.5	Aufstellen des linearen Gleichungssystems	4
2.6	Approximationsfehler	4
2.7	Rundungsfehler	4
3	Experimente	5
3.1	Die Approximationslösung und die exakte Lösung	5
3.2	Der Approximationsfehler	6
3.3	Speicherplatzbedarf der Matrix A	7
3.4	Speicherplatzbedarf der LU-Zerlegung	9
3.5	Approximation für $n = 4$ und $n = 11$	9
4	Zusammenfassung	10
5	Quellenverzeichnis	10
6	Dokumentation	11
7	experiments_lu.py	11
7.1	Purpose	11
7.2	Functions	11
8	block_matrix_2d.py	12
8.1	Purpose	12
8.2	Class: BlockMatrix	12
8.3	Additional Function	14
9	poisson_problem_2d.py	14
9.1	Purpose	14

9.2 Functions	14
10 linear_solvers.py	16
10.1 Purpose	16
10.2 Implemented	16
10.3 Placeholders	16
11 sparse_A_Matrix.py	17
11.1 Purpose	17
11.2 Functions	17
12 User manual	17

1 Einführung und Motivation

In diesem Handout untersuchen wir die Poisson-Gleichung mit Hilfe von finiten Differenzen. Es wurde das Konvergenzverhalten des Approximationsverfahrens für bestimmte Gleichungen untersucht.

Anhand der theoretischen Herleitung wurde das Verfahren in einem Programm mithilfe von Python algorithmisch implementiert und visualisiert. Weiterhin wurde der Approximationsfehler des Verfahrens theoretisch hergeleitet und anschließend in dem Programm implementiert und visualisiert.

Mit diesem Programm konnten wir numerische Experimente für bestimmte Funktionen durchführen, um das Konvergenzverhalten des Verfahrens zu prüfen. Dabei konnte man bestimmte Effekte beobachten, die sich mit den Approximationsfehlern und den Rundungsfehlern erklären lassen. Diese Effekte haben wir letztendlich in Kontext gesetzt und analysiert.

2 Theoretische Grundlagen

In diesem Handout sei ein Intervall $I \subset \mathbb{R}$ nicht einelementig.

2.1 Zweite finite Differenz

Definition 2.1 (Zweite finite Differenz). Sei $h \in \mathbb{R}$ mit $0 < h$, $x \in (a, b)$, $x + h \in [a, b]$ und $f : [a, b] \rightarrow \mathbb{R}$. Dann ist:

$$D_h^{(2)} f(x) := \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

die **zweite finite Differenz**.

Satz 2.2. Sei $0 < h \in \mathbb{R}$, $x \in (a, b)$, $x + h \in [a, b]$ und $f : [a, b] \rightarrow \mathbb{R}$ mit $f \in C^\infty([a, b])$. Dann gilt:

$$D_h^{(2)} f(x) = f''(x) + \mathcal{O}(h^2)$$

Poisson-Problem: Theorie, Diskretisierung und LGS

Der Laplace-Operator einer Funktion $u \in C^2(\mathbb{R}^2; \mathbb{R})$ ist definiert durch

$$\Delta u := \frac{\partial^2 u}{\partial x_1^2} + \frac{\partial^2 u}{\partial x_2^2}.$$

Für ein Gebiet $\Omega \subset \mathbb{R}^2$ mit Rand $\partial\Omega$, eine Funktion $f \in C(\Omega)$ und eine Randfunktion $g \in C(\partial\Omega)$ lautet das Poisson-Problem:

$$-\Delta u = f \quad \text{in } \Omega, \quad u = g \quad \text{auf } \partial\Omega.$$

Im Praktikum betrachten wir ausschließlich

$$\Omega = (0, 1)^2, \quad g \equiv 0.$$

Durch Diskretisierung ergibt sich ein lineares Gleichungssystem

$$A\hat{u} = b,$$

wobei $\hat{u} \in \mathbb{R}^N$ die Werte von u an den inneren Gitterpunkten approximiert.

2.2 Diskretisierung des Gebietes

Das Intervall $(0, 1)$ wird in $n \in \mathbb{N}$ Teilintervalle zerlegt. Die inneren Punkte sind

$$X_1 = \left\{ \frac{j}{n} : 1 \leq j \leq n-1 \right\}.$$

Für das Quadrat Ω ergibt sich

$$X = X_1 \times X_1 = \left\{ \left(\frac{j}{n}, \frac{k}{n} \right) : 1 \leq j, k \leq n-1 \right\}.$$

2.3 Diskretisierung des Laplace-Operators

Für $\ell = 1, 2$ sei e_ℓ der ℓ -te Einheitsvektor. Für $x \in X$ definiere man

$$u_{x,\ell}(t) := u(x + te_\ell).$$

Die zweite Ableitung wird durch finite Differenzen approximiert:

$$\frac{\partial^2 u(x)}{\partial x_\ell^2} = u_{x,\ell}''(0) \approx D_h^{(2)} u_{x,\ell}(0).$$

Damit lautet der diskrete Laplace-Operator:

$$\Delta_h u(x) := D_h^{(2)} u_{x,1}(0) + D_h^{(2)} u_{x,2}(0).$$

Für $h = 1/n$ liegt $x \pm he_\ell$ immer in $X \cup \partial\Omega$. Da $u = 0$ auf $\partial\Omega$, können Randpunkte direkt berücksichtigt werden.

2.4 Definition der Block-Diagonal-Matrizen

Sei $N = (n-1)^2$. Die Matrix $A \in \mathbb{R}^{N \times N}$, die beim 2D-Finite-Differenzen-Ansatz entsteht, besteht aus Blockmatrizen.

Die Matrix $C \in \mathbb{R}^{(n-1) \times (n-1)}$ führt zur Gesamtmatrix $A \in \mathbb{R}^{N \times N}$:

$$C := \begin{pmatrix} 4 & -1 & 0 & 0 & \cdots & 0 \\ -1 & 4 & -1 & 0 & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & -1 & 4 & -1 \\ 0 & \cdots & 0 & 0 & -1 & 4 \end{pmatrix} \quad A := \begin{pmatrix} C & -I & 0 & \cdots & 0 \\ -I & C & -I & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -I & C & -I \\ 0 & \cdots & 0 & -I & C \end{pmatrix}$$

Die Schrittweite lautet $h = 1/n$. Da A sehr dünn besetzt ist, lohnt sich die Speicherung als sparse Matrix.

2.5 Aufstellen des linearen Gleichungssystems

Die diskrete Gleichung lautet:

$$-\Delta_h u(x) = f(x), \quad x \in X.$$

Durch die beschriebene Approximation des Laplace-Operators ergeben sich N lineare Gleichungen. Um diese als lineares Gleichungssystem zu schreiben, muss jedem Diskretisierungspunkt $x \in X$ eindeutig eine Gleichungsnummer zugeordnet werden. Die resultierende Matrix ist $h^{-2}A$.

2.6 Approximationsfehler

Definition 2.3. Sei $\hat{u}$ die approximierte Lösung und u die exakte Funktion und u_i zu $\hat{u}_i$ der dazugehörige Funktionswert am gleichen Diskretisierungspunkt. Wir definieren den **Approximationsfehler** dann als:

$$e_u^{(n)} = \max_{i=1, \dots, (n-1)^2} |\hat{u}_i - u_i|$$

2.7 Rundungsfehler

Definition 2.4. Sei $p \in \mathbb{N}$. Die **Kondition** einer Matrix A wird durch

$$\text{cond}_p(A) = \|A^{-1}\|_p \|A\|_p$$

definiert.

Anmerkung: Im Fall $p = \infty$ ist die Norm einer Matrix als Maximumsnorm definiert. Die Subtraktion ist schlecht konditioniert[3], da gilt

$$\kappa = \frac{|a| + |b|}{|a + b|}$$

und daraus folgt, dass $|a + b| \ll |a| + |b| \iff \kappa \gg 1$.

3 Experimente

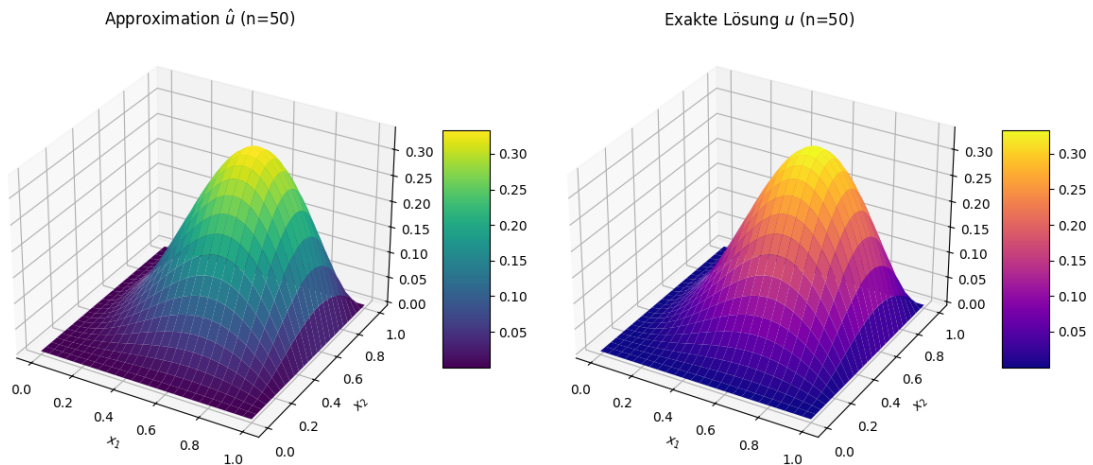
Für dieses Projekt haben wir in den Experimenten die Eigenschaften des Verfahrens untersucht und die Ergebnisse und Beobachtungen mit den theoretischen Herleitungen verglichen. Zu den untersuchten Eigenschaften gehören: die Konvergenz, die Konditionszahl von A , der Speicherplatzbedarf der Matrix A und der Speicherplatzbedarf der LU-Zerlegung von A .

Wir haben die Funktion $u : \Omega \rightarrow \mathbb{R}$ mit $u(x) = x_1 x_2 \sin(k\pi x_1) \sin(k\pi x_2)$ für ein $k \in \mathbb{N}$ gegeben. $f(x)$ ist gegeben durch

$$\begin{aligned} f(x) &= -\Delta u = \frac{\partial^2 u}{\partial x_1^2} + \frac{\partial^2 u}{\partial x_2^2} \\ &= 2kx_2 \sin(kx_2) \cos(kx_1) - 2k^2 x_1 x_2 \sin(kx_2) \sin(kx_1) + 2kx_1 \sin(kx_1) \cos(kx_2) \end{aligned}$$

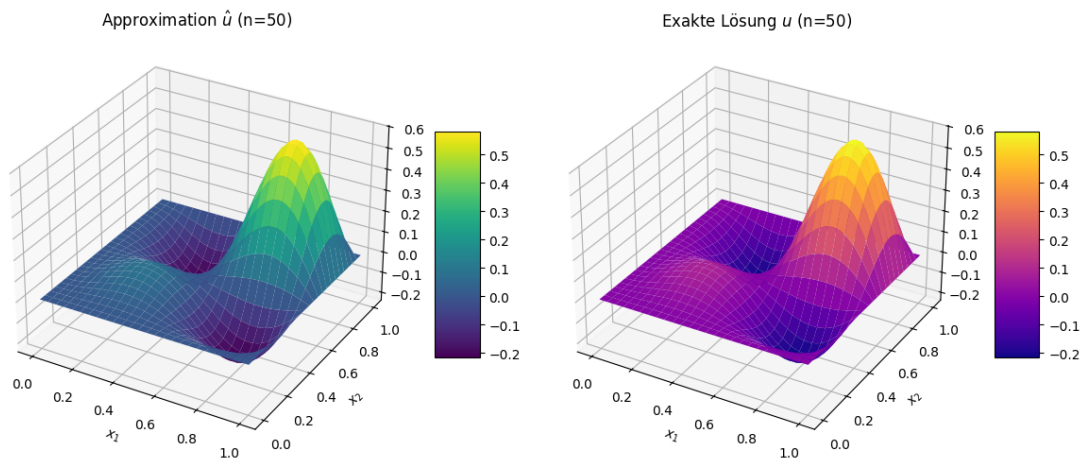
3.1 Die Approximationslösung und die exakte Lösung

Wir betrachte zuerst den Fall $k = 1$:



In den Experimenten untersuchen wir die oben genannten Eigenschaften für jeweils $k = 1$ und $k = 2$. Zum Beginn ist es interessant, sich die Grafiken der Funktionen anzusehen. Deshalb stellen wir die approximierte und exakte Lösung gegenüber.

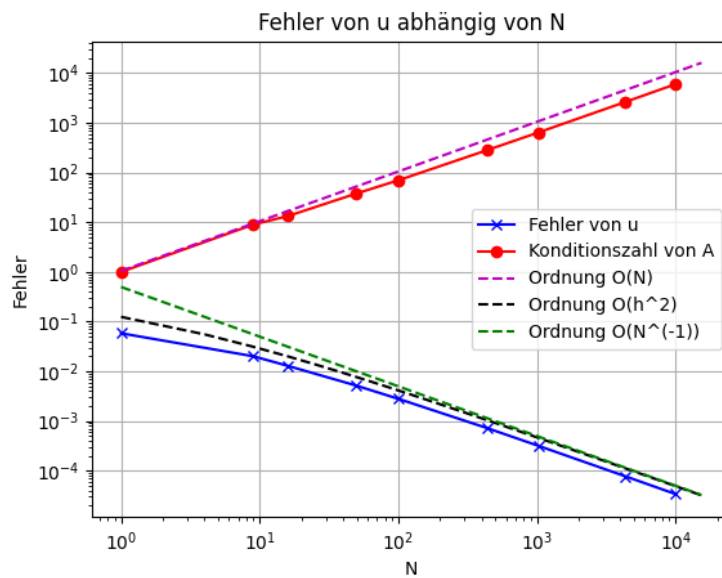
Wir betrachten danach den Fall $k = 2$:



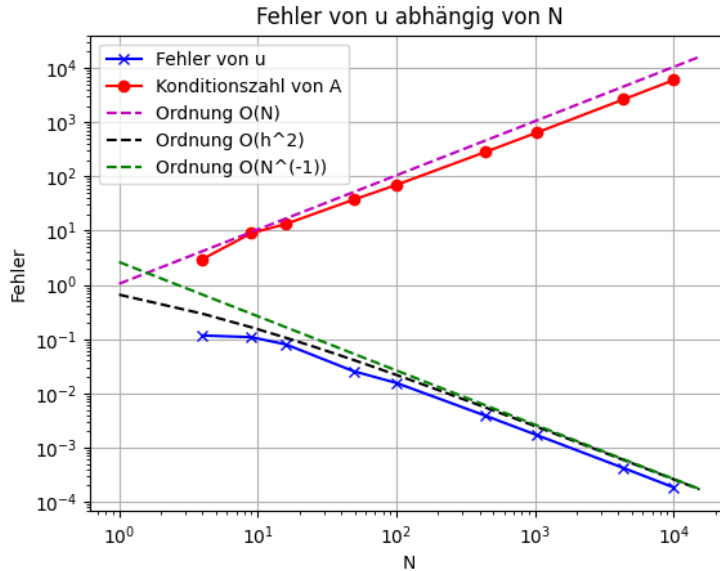
Wir können feststellen, dass diese Methode funktioniert, da die beiden Grafen ziemlich ähnlich aussehen und deshalb nichts über den Fehler sagen können.

3.2 Der Approximationsfehler

Für $k = 1$:



Für $k = 2$:



Wir In beiden Fällen konvergiert der Approximationsfehler mit $\mathcal{O}(h^2)$, d.h. der Approximationsfehler der zweiten finiten Differenz dominiert den Fehler. Die Kondition der Matrix A bezüglich der Maximumsnorm wächst dafür linear mit N oder $\mathcal{O}(h^{-2})$. Damit die Kondtion also eine Auswirkung hat, muss N Richtung Maschinengenauigkeit gehen, dies ist jedoch noch nicht der Fall bei unserem Fehler. Dennoch entwickelt sich die Kondition mit $\mathcal{O}(N)$, das bedeutet, dass die Kondition ziemlich schnell größer als 1 wird, also wenn man dieses Approximationsverfahren mit experimentell ermittelten Daten benutzt, sollte man vorsichtig bei der Auswahl von n sein.

3.3 Speicherplatzbedarf der Matrix A

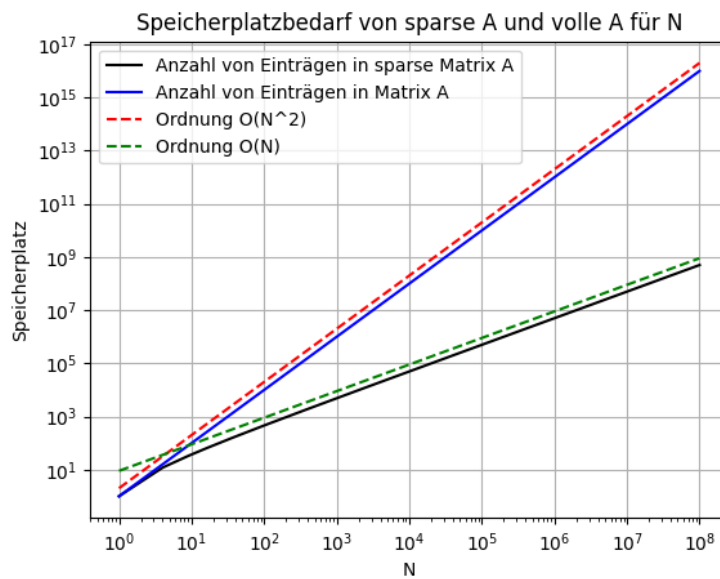
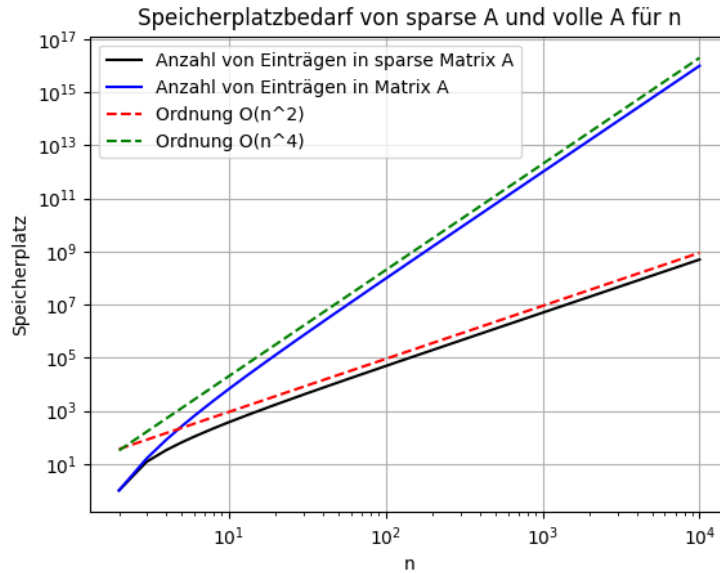
Wir untersuchen den Speicherplatzbedarf der Matrix A im *sparse* Format an und vergleichen ihn mit einer vollbesetzten Matrix. im *sparse* Format kann man di-, bi- und triagonale Matrizen, aber auch Matrizen mit Diagonalen links und rechts der Hauptdiagonale effiezent speichern. Das Format speichert nur die Werte auf diesen Diagonalen.

In unserem Fall können wir die Matrix A zusammenstellen aus den $-I$ und C Matrizen. $-I$ lässt sich als diagonale Matrix im *sparse* Format darstellen lassen, man speichert nur die Werte auf der Diagonalen und C speichern wir als eine tridiagonale Matrix auch im *sparse* Format.

Definition 3.1. Die Anzahl der nicht-Nullelemente einer Matrix M wird mit $\#(M)$ bezeichnet.

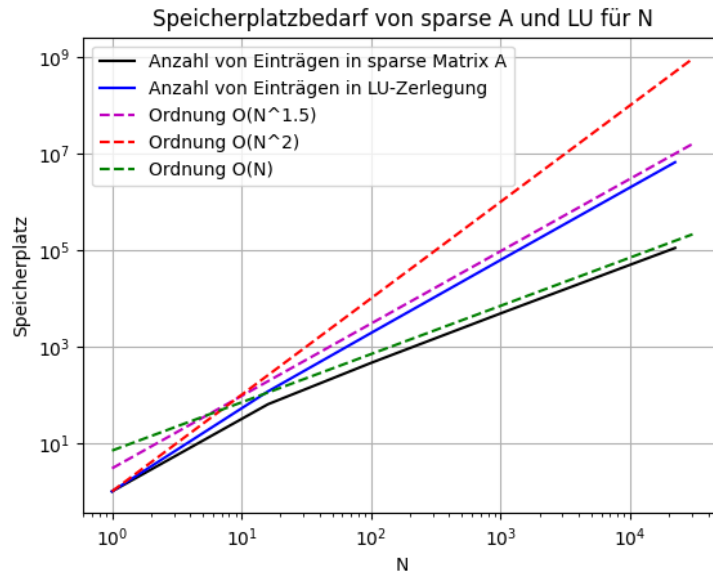
Die Anzahl der nicht-Nullelemente der Matrix I ist gleich $\#(I) = (n - 1)$ und $\#(C) = (n - 1) + 2(n - 2)$. Daraus folgt, $\#(A) = (n - 1)\#(C) + 2(n - 2)\#(I) = 5(n - 1)^2 - 4(n - 1) = 5N - 4\sqrt{N}$.

Weiterhin kann man beobachten, dass der Fehler unabhängig davon ist ob $k = 1$ oder $k = 2$.



Wie man sieht ist das *sparse* Format viel effizienter bei der Matrix A , deshalb kann man mit größeren Matrizen arbeiten.

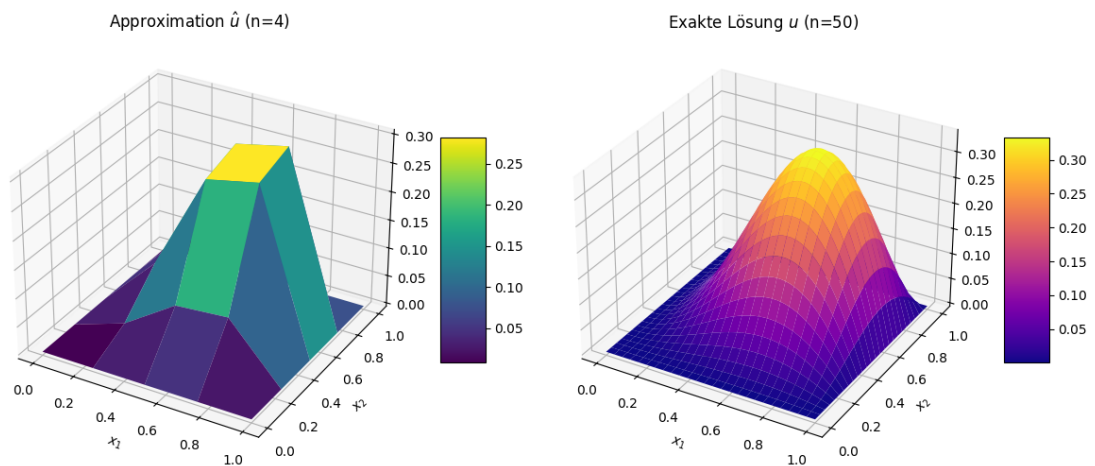
3.4 Speicherplatzbedarf der LU-Zerlegung

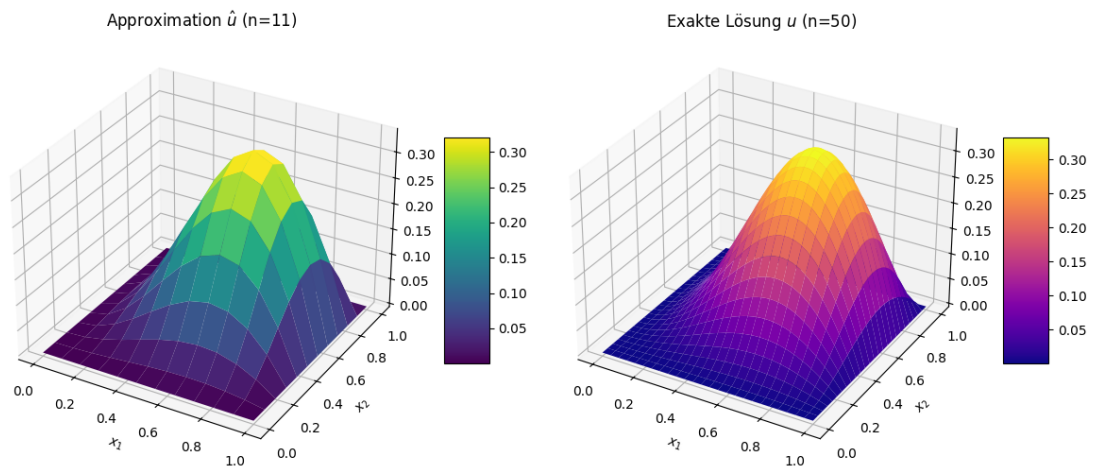


Wie wir sehen wächst die Anzahl der nicht Null-Einträge bei der LU-Zerlegung mit $\mathcal{O}(N^{-1.5})$, das kommt davon, dass wenn man sich die Matrizen L und U anschaut, immer $1 + 2(n - 1)$ Diagonalen besetzt sind, also durch aufsummieren der Anzahl der Elemente bekommt man $\#(L) + \#(P) = (n - 1)^2 + 2(n - 1)(n - 1)^2 - n(n - 1) = N + 2N^{1.5} - N + \sqrt{N} = 2N^{1.5} + \sqrt{N}$.

Unser Verfahren ist leider aber ziemlich begrenzt, da die LU-Zerlegung im Programm als einfacher Array gespeichert wird, mit allen Einträgen. also bei $N = 1000000$ oder $n = 1000$ braucht man schon $10^{12} \times 8$ Bytes oder auch 8000 Gigabytes Speicher.

3.5 Approximation für $n = 4$ und $n = 11$





4 Zusammenfassung

Im Rahmen unseres Projekts haben wir sowohl bereits vorhandenes Wissen, wie Kenntnisse über Finite-Differenzen, als auch Wissen aus AB2 angewendet: die Lösung des Poisson-Problems mithilfe numerischer Methoden.

5 Quellenverzeichnis

Literatur

- [1] Hella Rabus, *Vorlesung Einführung in das wissenschaftliche Rechnen*, Sommersemester 2025
- [2] Otto Forster, *Analysis 1: Differential- und Integralrechnung einer Veränderlichen*, 11. Auflage, Springer Spektrum, 2013
- [3] Peter Deufelhard, Andreas Hohmann, *Numerische Mathematik 1: Eine algorithmisch orientierte Einführung*, 4. Auflage, de Gruyter, 2008

Übersicht verwendeter Hilfsmittel

Wir versichern, dass wir bei der Erstellung dieser Studienarbeit keinerlei IT-/KI-gestützte Schreibwerkzeuge verwendet haben.

6 Dokumentation

This project implements a numerical method for solving the two-dimensional Poisson equation with boundaries set at zero using finite differences. It is accomplished by the assembly of a Matrix A , a sparse representation of A , LU decomposition, iterative solvers, error computation, and visualizations. There are 5 different files:

- `experiments_lu.py` – main interface to run experiments
- `block_matrix_2d.py` – construction and LU decomposition of the matrix A of the linearization
- `poisson_problem_2d.py` – computation of the numerical solution, visualization and error computation
- `linear_solvers.py` – Linear equations solver with given LU decomposition
- `sparse_A_Matrix.py` – analytical memory calculation

7 experiments_lu.py

7.1 Purpose

Interactive script to run the experiments, compare numerical and exact solutions, and analyze memory use.

7.2 Functions

`exact_u_func(x,  $\kappa = 1$ )`

Input: x (array of floats) represents the point in the space for which the solution is evaluated

κ (int) parameter in the source term

Returns: (float) analytical solution of the two-dimensional Poisson equation

Analytical solution: $u(x) = x_1 x_2 \sin(\kappa \pi x_1) \sin(\kappa \pi x_2)$.

`f_source(x,  $\kappa=1$ )`

Input: x (array of floats) represents the point in the space for which the solution is evaluated

κ (int) parameter in the source term

Returns: laplace (float) the value of the function $f(x)$

Right-hand side computed from $f = -\Delta u$.

`main()`

Offers multiple experiment modes:

- 1 Funktion u und die approximierte Lösung

- 2 Approximationsfehler für verschiedene N
- 3 Speicherplatzbedarf von sparse A und LU für N
- 4 Speicherplatzbedarf von sparse A und volle A für n
- 5 Speicherplatzbedarf von sparse A und volle A für N
- 6 Ableitungsapproximationen für f'' für mehrere h -Werte
- 0 Beenden

8 block_matrix_2d.py

8.1 Purpose

Input: n (int) number of intervals in each dimension

Raises: (Value Error) if $n < 2$

Constructs a sparse block matrix for the discrete Laplacian using the 5-point stencil, with analysis tools.

8.2 Class: BlockMatrix

Represents matrix with grid size n and $N = (n - 1)^2$ unknowns.

Attributes

- n (int) number of intervals in each dimension
- N (int) total number of interior grid points, calculated as $(n - 1)^2$ used to compute total matrix size
- A_sparse (scipy.sparse.csr_matrix) sparse representation of the block matrix. Initially None, computed later on
- L (numpy.ndarray) lower-triangular unit-diagonal factor of A
- U (numpy.ndarray) upper-triangular factor of A
- P (numpy.ndarray) permutation matrix from LU decomposition of A

All above mentioned attributes are non-static.

Methods

- `get_sparse()` – constructs sparse matrix

Input: self (Blockmatrix object) containing:

self. n (int) number of intervals in each dimension

self. A_sparse (scipy.sparse.csr_matrix) cached sparse matrix

Returns: self.A_sparse (scipy.sparse.csr_matrix) sparse CSR matrix representing the finite difference block matrix for the two-dimensional Laplace Operator

- **eval_sparsity()** – returns number of non-zero elements

Input: self (Blockmatrix object) containing:

self.N (int) total number of interior grid points $(n - 1)^2$ used to compute total matrix size

self.get_sparse() (self.A_sparse (scipy.sparse.csr_matrix)) sparse matrix whose sparsity is evaluated

Returns: nnz (int) number of non-zero elements in the sparse matrix

rel_nnz (float) ratio of non-zero elements to total possible elements in the full $N \times N$ matrix

- **get_lu()** – LU decomposition

Input: self (Blockmatrix object) containing:

self.A_sparse (scipy.sparse.csr_matrix) sparse matrix to be factorized

self.P, self.L, self.U (numpy.ndarray) cached LU-decomposition components

Returns: L (numpy.ndarray) lower-triangular unit-diagonal factor of A

U (numpy.ndarray) upper-triangular factor of A

P (numpy.ndarray) permutation matrix from LU decomposition of A

- **eval_sparsity_lu()** – nonzeros in L and U

Input: self (Blockmatrix object) containing:

self.n (int) number of intervals in each dimension

self.N (int) total number of interior grid points $(n - 1)^2$ used to compute total matrix size

self.get_lu (p, l, u)

Returns: nnz (int) number of non-zero elements in the sparse matrix

rel_nnz (float) ratio of non-zero elements to total possible elements in the full $N \times N$ matrix

- **show_nonzeros()** – sparsity visualization

Input: self (Blockmatrix object) containing:

self.get_sparse (A (scipy.sparse.csr_matrix)) sparse matrix whose sparsity pattern is plotted

Returns: the method does not return a value

Output is a matplotlib figure showing the non-zero pattern of matrix.

- `get_cond()` – condition number

Input: `self` (Blockmatrix object) containing:

`self.get_sparse (A (scipy.sparse.csr_matrix))` sparse matrix whose condition number is computed

Returns: `cond (float)` condition number of the matrix `A`

8.3 Additional Function

`plot_LU_A_N(n)` – plots sparsity scaling for A and its LU factors.

Input: `n (int)` number of intervals in each dimension

Returns: Output is a matplotlib chart comparing sparsity of A , sparsity of $LU(A)$, curves $O(N)$ and $O(N^2)$

9 poisson_problem_2d.py

9.1 Purpose

Contains right-hand side assembly, numerical solution, error computation, and visualization.

9.2 Functions

- `rhs(n, f,  $\kappa = 1$ )` – constructs right-hand side vector on interior nodes

Input: `n (int)` number of intervals in each dimension

`f (callable)` right-hand-side function of the Poisson problem with signature $f(x, kappa)$

`x (numpy.ndarray of floats)` point with two coordinates

`kappa (int)`

Returns: `b(numpy.ndarray of floats)` right-hand-side vector of length $(n - 1)^2$, each element is $f([j/n, k/n], \kappa)$ evaluated at interior grid points

Raises: (Value Error) if $n < 2$

- `idx(nx, n)` and `inv_idx(m, n)` – index mappings

Input: `nx (list of int)` coordinates of a discretization point, multiplied by `n`
`n (int)` number of intervals in each dimension

Returns: `index (int)` number of the corresponding equation in the Poisson problem

Input: m (int) number of an equation in the Poisson Problem
 n (int) number of intervals in each dimension
Returns: $[nx0, nx1]$ (list of int) coordinates of the discretization point corresponding to equation m , multiplied by n

- `compute_error(n, û, u)` – computes max-norm error

Input: n (int) number of intervals in each dimension
 $\hat{u}$ (array_like of floats (numpy.ndarray))
 u (callable) exact solution of the Poisson problem with signature $u(x, \kappa)$
 x (numpy.ndarray of floats) point with two coordinates
 κ (int)
Returns: `max_error` (float) maximal absolute difference between the numerical solution $\hat{u}$ and the exact solution u at all interior points

- `plot_error(n, u, f, logscale = True,  $\kappa = 1$ )` – error and condition plots

Input: n (int) number of intervals in each dimension
 $\hat{u}$ (array_like of floats (numpy.ndarray))
 u (callable) exact solution of the Poisson problem with signature $u(x, \kappa)$
 x (numpy.ndarray of floats) point with two coordinates
 κ (int)
 f (callable) right-hand-side function of Poisson problem used to compute numerical solutions
`logscale` (bool, default=True) used to plot both axes on a logarithmic scale
 κ (int) parameter
Returns: Output is a matplotlib chart showing maximal errors of the numerical solution (`error_list`), condition numbers (`condition_list`), reference curves $O(N)$, $O(h^2)$, $O(N^{-1})$

- `compute_u(n, f,  $\kappa=1$ )` – assembles and solves $Au = b$

Input: n (int) number of intervals in each dimension
 f (callable) right-hand-side function of the Poisson problem with signature $f(x, \kappa)$
 x (numpy.ndarray of floats) point with two coordinates
 κ (int)
Returns: u (numpy.ndarray of floats) numerical solution of the Poisson problem at interior points reached by solving the linear system using LU decomposition

- `plot_solution(n, u,  $\kappa = 1$ )` – surface plot of exact solution

Input: n (int) number of intervals in each dimension
 u (callable) solution of the Poisson problem with signature $u(x, y)$ returns scalar value for each point in the grid κ (int) parameter
Returns: Output is a matplotlib 3D surface plot of the solution $u(x, y)$ in $[0, 1]^2$

- `plot_surface_comparison(n, u_func, f,  $\kappa = 1$ )` – numerical vs exact comparison

Input: n (int) number of intervals in each dimension
 u_func (callable) exact solution of the Poisson problem with signature $u_func([x1, x2], \kappa)$ returns scalar value
 f (callable) right-hand-side function of the Poisson problem used to compute numerical solutions
 κ (int)
Returns: Output is a matplotlib 3D chart comparing numerical approximation and exact solution

10 linear_solvers.py

10.1 Purpose

Contains linear solvers for the Poisson problem.

10.2 Implemented

`solve_lu(p, l, u, b)` – solves $Ax = b$ using LU factors.

Input: p (numpy.ndarray) permutation matrix from LU decomposition of A
 l (numpy.ndarray) lower-triangular unit-diagonal factor of A
 u (numpy.ndarray) upper-triangular factor of A
 b (numpy.ndarray of floats) right-hand-side vector of the linear system $Ax = b$
Returns: x (numpy.ndarray of floats) solution of the linear system $Ax = b$

10.3 Placeholders

SOR, Gauss–Seidel, Jacobi, and Conjugate Gradient solvers.

11 sparse_A_Matrix.py

11.1 Purpose

Provides analytical memory formulas for sparse and dense matrices.

11.2 Functions

- `compute_sparse_memory(n)` – number of sparse nonzeros
- `compute_sparse_memory_N(N)` – sparse memory vs. unknown count
- `compute_full(n)` – dense matrix entries
- `compute_full_N(N)` – dense memory vs. N
- plot functions for memory growth

12 User manual

```
def main():
    #n = 10
    #kappa = 1.0

    #bm = BlockMatrix(n)

    print("Wähle ein k")
    k = float(input("kappa = ").strip())

    print("\nWählen Sie aus, welches Diagramm angezeigt werden soll:")
    print("1 - Funktion u und die approximierte Lösung "
          "(plot_surface_comparison)")
    print("2 - Approximationsfehler für verschiedene N "
          "(plot_error)")
    print("3 - Speicherplatzbedarf von sparse A und LU für N "
          "(plot_LU_A_N)")
    print("4 - Speicherplatzbedarf von sparse A und volle A für n "
          "(show_n_log)")
    print("5 - Speicherplatzbedarf von sparse A und volle A für N "
          "(show_N)")
    #print("6 - Ableitungsapproximationen für f'' für mehrere h-Werte "
    #      "(plot_functions_list_2)")
    print("0 - Beenden")

    choice = input("Your choice: ").strip()

    if choice == "1":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        plot_surface_comparison(n, exact_u_func, f_source, kappa=k)
    elif choice == "2":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        plot_error(n, exact_u_func, f_source, kappa=k)
    elif choice == "3":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        plot_LU_A_N(n)
    elif choice == "4":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
```

```

        show_n_log(n)
    elif choice == "5":
        print("\nFür welches N möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        show_N(n)

    #elif choice == "6":

    elif choice == "0":
        print("Exiting program.")
    else:
        print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()

```

The standard output therefore produces the following output:

```

Wählen Sie aus, welches Diagramm angezeigt werden soll:
1 - Funktion u und die approximierte Lösung (plot_surface_comparison)
2 - Approximationsfehler für verschiedene N (plot_error)
3 - Speicherplatzbedarf von sparse A und LU für N (plot_LU_A_N)
4 - Speicherplatzbedarf von sparse A und volle A für n (show_n_log)
5 - Speicherplatzbedarf von sparse A und volle A für N (show_N)
0 - Beenden
Your choice:

```